

Member-only story

I Turned Claude Code Into My Chief of Staff (One Folder, 6 Skills)

Everyone aims Claude Code at their repo. I aimed it at my week. The folder that came out runs my mornings, and it taught me the two things that decide whether one of these personal AI workspace setups compounds or quietly dies in a week.



Anubhav

Follow

12 min read · 4 days ago



126



1



1



Read the article for free [here](#).

Anubhav highlighted



For about four months, every working session with Claude Code started the same way. I'd paste the same 200 words before any real work could start — who I am, what I'm shipping this quarter, and what happened yesterday. I was the context window's unpaid intern.

Then one morning I typed six characters, `/standup`, and it already knew.

Yesterday's three open threads. Today's two meetings, pulled from my real calendar. The decision I had parked on Tuesday and told myself I would revisit.

The trick is not a complex prompt. It is a folder. One directory called `HQ/` that holds a `CLAUDE.md` teaching Claude who I am, six skills, and a connection to my calendar, email, and Slack through MCP.

But the folder is the easy twenty percent. Whether it becomes something that runs your day or a tidy notes template you abandon by Friday comes down to two specific conditions. We will build the folder first. Then I will show you both.

Claude Code Was Never Really A Coding Tool

It feels like a coding tool because that is where it launched. Developers pointed it at their repositories and asked it to write tests, debug React components, or refactor legacy Python scripts. Because it runs in the terminal and executes bash, we assumed its primary identity was a junior developer.

But underneath the surface, it is a filesystem agent. It is a loop that can read files, write files, search directories, and reach connected tools. Code was just the first thing aimed at it.

The exact same four primitives make it work for non-code tasks. You have `CLAUDE.md` for baseline context, skills for saved procedures, MCP for external connections, and plain-text memory for the historical record.

When you stop asking what coding skill to install next, you start asking what you are re-explaining to a model every single day. The highest return on your time in mid-2026 is pointing the agent at your actual work with a searchable trail and a few real tools.

The One Folder

To make this real, we need a place for the agent to live. The entire chief of staff setup is contained within a single directory.

```
mkdir -p HQ/{log,meetings,decisions,drafts} HQ/.claude/skills  
cd HQ  
# CLAUDE.md and .mcp.json get created in the next steps
```

The `.claude/skills/` directory sitting inside `HQ/` is doing something deliberate: project scoping. By putting the skills folder here rather than in your global `~/.claude/` directory, the skills travel with the folder. If you move this folder to a new laptop, your entire operational setup comes with it.

Each subdirectory maps to a specific kind of work: `log/` for daily execution notes, `meetings/` for transcripts and prep, and `decisions/` as an append-only ledger of why you did what you did.

The CLAUDE.md That Knows You

A `CLAUDE.md` file is the document Claude reads automatically at the start of every session. In a software repository, it usually holds build commands, testing standards, and architecture rules. In your personal headquarters, it holds you.

It contains your identity, your current workstreams, how you want the agent to work, and your writing voice.

```
# Who I am
Technical lead on the platform team. I write code and manage architecture.
I think on the page so expect half-formed notes rather than polished briefs.

# How to work with me
Lead with the answer. Default to short. Flag when you are guessing.
When I dump something raw, file it. Do not ask twenty questions first.

# Current workstreams <!-- /wrap rewrites this block nightly -->
- Database migration blocking on security review
- Q3 planning with early infrastructure drafts
- Hiring pipeline for the new backend engineer

# My voice
Plain, direct, lowercase-leaning. No exclamation marks.
```

The real file is about 40 lines long, but this trimmed version shows the load-bearing blocks. The payoff is not the content itself, it's the loading

mechanism. This is the exact context you used to paste manually, now entering the context window on its own every single session without any effort.

A static `CLAUDE.md` is right for a week and wrong by the end of the month. The fix is not pure willpower. The fix is having your evening `/wrap` command rewrite that specific block for you, which we will build shortly.

The Six Skills

Six skills cover the whole day. The number matters less than the rule behind it: every skill you keep installed rides in the model's context on each turn, whether it fires or not — the plugin marketplace even prints that per-turn cost next to each one. So a personal setup should hold only the handful you actually touch every day.

The invocation split is the one design decision worth understanding deeply. Skills are model-invoked by default. Claude reads the description in the background and fires the skill when it judges the moment is right. If you set the `disable-model-invocation` field to true in the YAML frontmatter, it becomes slash-only.

Most of these skills belong under my explicit control. A morning brief that fires itself randomly while I am trying to write a document is highly annoying.

The `/meeting` skill is the deliberate exception. Its description tells Claude to offer preparation material when I mention an upcoming call, and to

automatically file notes when I paste a transcript. That is the one place I want the chief of staff leaning in without being asked.

The others are strictly manual. I typed `/dump` mid-call last week to save a stray thought about pricing, and it filed it to the daily log without interrupting the meeting flow.

I left the once-a-month work out on purpose — competitive scans, formal decision logs, stakeholder updates. They aren't daily, and daily repetition is the only thing that turns this from a clever setup into a habit you keep. Add one back when a real need shows up.

One Skill All The Way Down: `/standup`

A skill is just a file. It is a few lines of YAML saying when to use it, followed by plain text instructions. The directory name is the command itself.

This file lives at `HQ/.claude/skills/standup/SKILL.md`.

```
---
name: standup
description: Morning brief. Use when I type /standup or ask for a morning
  briefing. Reads yesterday's log and today's calendar.
disable-model-invocation: true
allowed-tools: Read Bash(ls *) mcp__gcal-server__*
```

```
---  
  
# Standup  
1. Read the newest file in `log/` for yesterday's open threads and carry-forward  
2. Pull today's events from Google Calendar (via the connected MCP server).  
3. Scan `decisions/` for anything tagged "#revisit".  
4. Write a brief to the top of a new `log/YYYY-MM-DD.md`:  
    - Today's calendar, with times  
    - Carrying over: unfinished threads from yesterday  
    - Top 3: the highest-leverage things to move today  
Keep it under 200 words. No filler, no pep talk.
```

Claude only pulls a skill's full text into context when it is triggered. Six skills cost almost nothing at rest because the system only loads their one-line descriptions initially. This is progressive disclosure. It is also why the exact same file runs unmodified in Codex, Gemini CLI, and Cursor. Skills are the portable layer.

The `allowed-tools` field is the guardrail. This CLI-only field restricts what the skill can do. It can read files, list directory contents, and reach the calendar tool. But it lacks the `write` permission, meaning your morning standup cannot accidentally overwrite your historical logs. MCP tools are allowlisted by name too, which is why `mcp__gcal-server__*` is explicitly included.

Step two only works once we wire the connectors. It tells the agent to pull today's events from Google Calendar.

The Engine That Feeds It: /wrap

Before we wire the calendar, we have to look at the engine that actually feeds this entire system. The morning standup is useless if it has nothing to read. The `/wrap` skill is the backbone of the argument. It is the file that creates the history, tags the decisions, and rewrites the context.

This file lives at `HQ/.claude/skills/wrap/SKILL.md`.

```
---
name: wrap
description: End of day routine. Sorts the daily log, extracts decisions,
  and updates the CLAUDE.md workstreams block.
disable-model-invocation: true
allowed-tools: Read Write Bash(ls *) mcp__slack-server__*
---

# Wrap Routine
1. Read today's file in `log/`.
2. Identify any final decisions made today. Move them to a new file in
  `decisions/YYYY-MM-DD-topic.md`. If a decision needs future review,
  append the tag "#revisit" to the bottom of that file.
3. Identify open threads and leave them at the bottom of today's log
  under a "Carry Forward" header.
4. Read `CLAUDE.md`. Look at the "Current workstreams" block. Based on
  today's log, rewrite that specific block to reflect reality. Overwrite
  the file with the updated block.
5. Post a 3-bullet summary of today's progress to the Slack team channel
  (via the connected MCP server).
```

This is how the system stays alive. When you run `/wrap` at 8:00 PM, the agent scans your raw daily log. It pulls out the hard decisions and isolates them. It explicitly tags uncertain choices with `#revisit`, which closes the loop for step three of the `/standup` skill we wrote earlier.

Most importantly, it rewrites the `CLAUDE.md` file. You do not have to manually update your current priorities. The agent reads what you actually worked on and updates its own core instructions for the next morning.

Step 5 posts to Slack. Like the calendar line in `/standup`, that depends on a connector we wire next.

Proving The Connectors: /triage

To see exactly why the connectors matter, look at `/triage`. This skill bridges the gap between your local files and your live communication.

This file lives at `HQ/.claude/skills/triage/SKILL.md`.

```
---
name: triage
description: Scans unread Slack mentions and drafts bulleted replies.
disable-model-invocation: true
allowed-tools: Read Write mcp__slack-server__*
---

# Triage Routine
1. Fetch my unread mentions from Slack (via the connected MCP server).
2. For each mention requiring a response, draft a short, direct reply matching m
3. Save these drafts into `drafts/slack-replies-YYYY-MM-DD.md`.
4. Do NOT send the messages. Only draft them locally for my review.
```

The `allowed-tools` field gives it write access to the local filesystem and read

access to Slack. The instruction in step 4 ensures it drafts only and never sends. This is the safety point that makes the skill usable.

The other three skills follow this exact same pattern. If you understand `/standup`, `/wrap`, and `/triage`, you understand the architecture.

Wiring The Tools Is Where It Stops Being A Filing Cabinet

Condition one: the connectors have to be real.

Without connectors, the whole setup is just a tidy notes template. The calendar step in the standup skill will write a confident brief over an empty calendar. You will trust it until you miss a critical meeting. We need to connect the folder to the outside world.

In mid-2026, the MCP ecosystem has matured. You do not have to write custom integration code. You can use standard MCP servers provided by hosted platforms or deploy official open-source servers locally.

You authenticate through their dashboard, link your Slack workspace and Google Calendar via OAuth, and they provide a secure endpoint and a bearer token. Here is how you actually wire them into Claude Code.

```
# Run from inside HQ/ so the servers land in HQ/.mcp.json (project scope)

# 1. Add the Slack connector via HTTP transport (using your hosted MCP provider)
claude mcp add slack-server --transport http https://api.composio.dev/mcp/slack
--header "Authorization: Bearer $COMPOSIO_API_KEY" --scope project

# 2. Add the Google Calendar connector
claude mcp add gcal-server --transport http https://api.composio.dev/mcp/googlec
--header "Authorization: Bearer $COMPOSIO_API_KEY" --scope project

# Restart the session for Claude Code to load the new servers.
```

The `--scope project` flag isolates the access. It writes the server configuration to a `.mcp.json` file directly inside the `HQ/` folder rather than polluting your global `~/.claude/mcp.json` config. If you open Claude Code in a different coding project, it will not have access to your calendar. The access stays strictly bound to the headquarters.

Where It Breaks

Connectors drop. It will work perfectly once and then silently fail during the

next session. When that hits your morning standup, you do not get a error. You get a confident brief based on stale or empty data. That is much worse than having no brief at all. Until the session state is completely solid, you have to eyeball the calendar block and ask yourself if it is real.

Multi-account routing is rough. Having a personal Gmail and a work Gmail in one workspace has no clean first-party story yet. You have to pick the account that holds your real work and accept the blind spot for the other.

Those are the technical bugs. The behavioral bug is condition two: **you have to feed it.**

The morning standup is only as good as last night's wrap command. If you skip the end of day pass for a few days, the brief has nothing to carry forward. The decisions folder goes quiet. The workstreams in your `CLAUDE.md` drift out of sync with reality. No skill design fixes this. It is a daily habit wearing a technical costume.

The Part That Actually Compounds

After about two weeks of actually running the wrap command, the log and decisions folders stop being a diary and become a searchable history.

The concrete moment happens when you ask what you decided about the

database migration rollback steps back in May and why you dropped the automated script. Claude reads three files out of `decisions/` and `log/` and answers from what you wrote at the time, including the objection you talked yourself out of and then hit anyway. That is not a guess. It is your own trail.

The moat is plain text in a folder you own and can grep, not a vector database you cannot inspect or a service that holds your data hostage.

The skills were never the point. They are just the rhythm that keeps feeding the folder. Every wrap command adds a day. The real value is that six months from now, the reason why you did something a certain way lives in your folder instead of the gap between four people's memories.

The Two Conditions

Anubhav highlighted

So it comes down to two things: the connectors have to be real, and you have to feed the folder. Miss either and you've built a tidy template.

Anubhav highlighted

Most of these setups die by Friday because they were built to be impressive, not to be run. The one that survives is the smaller one you open every morning and close every night. Build the six skills, wire the two connectors, and run `/wrap` tonight. Give it a week and the folder stops being a template. It becomes the thing that remembers what you decided, and why, long after you have.

Anubhav highlighted

Anubhav highlighted

Continue Reading

[The 8 Skills Every Claude Code Setup Needs in 2026](#): Extend the same skills pattern into a stronger Claude Code setup.

[The 8 MCP Servers Every Claude Code Setup Needs in 2026](#): Add real calendar, Slack, browser, and workspace connectors safely.

[If You're Still Hitting the Claude Code 5-Hour Wall, You're Doing It Wrong](#): Fix the workflow habits that quietly waste Claude Code sessions.

[What Is MCP? Build a Custom MCP Server in Python](#): Understand the connector layer powering Claude Code beyond local files

Machine Learning

Deep Learning

Artificial Intelligence

AI

Programming



Published in Data Science Collective

932K followers · Last published 16 hours ago

Follow

Advice, insights, and ideas from the Medium data science community



Written by Anubhav

3.4K followers · 10 following

Follow

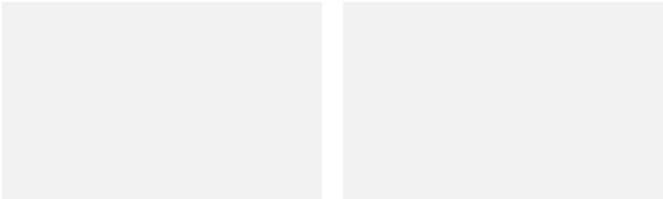
I write deep-dives on agents, RAG, evals, and what actually breaks when you ship AI systems



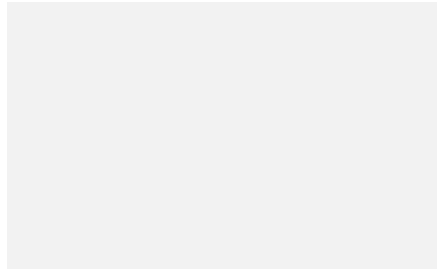
Placeholder for author name

Placeholder for article title

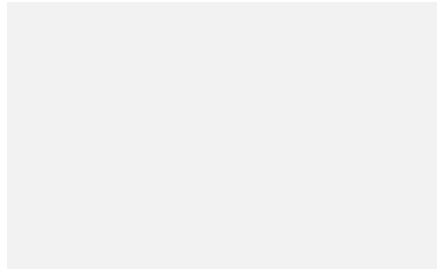
Placeholder for article description



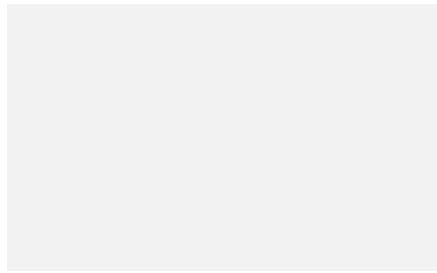
Placeholder for article tags



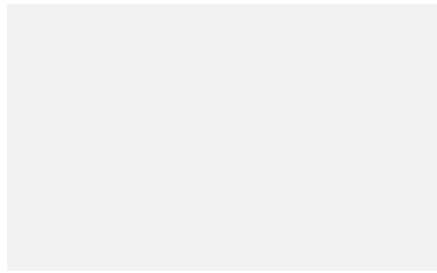
[Redacted text block]



[Redacted text block]



[Redacted text block]



[Redacted text block]